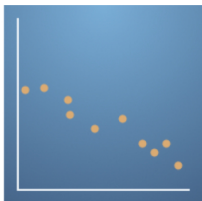
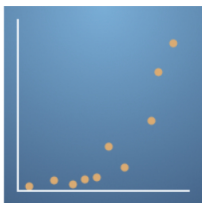
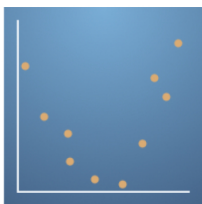
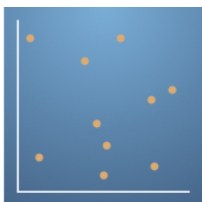


1. In the previous video you saw how to fit a line $y = mx + c$ to linear data. In this quiz you will practise identifying data that is appropriate for linear regression, and initialise some fits yourself.

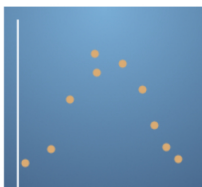
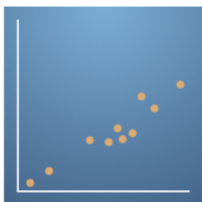
1 point

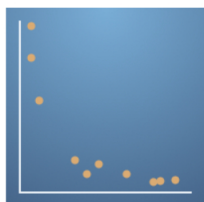
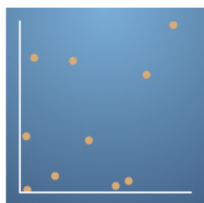
Which of the following figures looks like it contains sensible data for a linear fit?



2. Which of the following figures looks like it contains sensible data for a linear fit?

1 point





3. Now that we've identified candidates for linear regression we can do some linear fitting ourselves. The code block below plots some predefined data points and a linear regression with the values $[m, c]$, where m is the gradient and c is the y-intercept. It also gives the χ^2 value discussed in the previous video, which is a measure of how good the fit is.

1 point

Play with the values of m and c to get a sense for how different linear fits affect χ^2 , then try to find the best possible fit to the data.

```
1 # See what m and c do to the fit
2 m = -0.24 ; c = 0.78
3 p = [m,c]
4 line(p)
```

Run

Reset

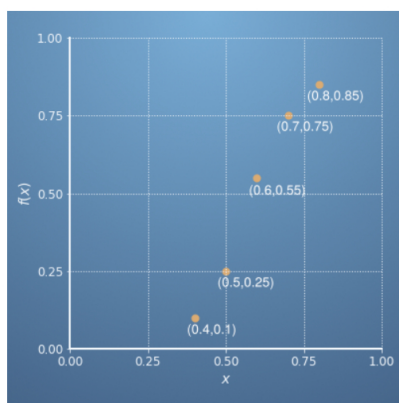
The minimum χ^2 value is 0.03819 to 4 significant figures. Try to find a fit with $\chi^2 \leq 0.04$ and then input these values into the following code block:

```
1 # Replace m and c with values that minimise  $\chi^2$ .
2 p = [-0.24, 0.78]
```

4. Fitting by eye is not that easy even for small sets of data. Let's make some linear fits using the maths discussed in the previous video.

1 point

The following is a figure with 5 data points, labelled with their (x, y) coordinates:



Let's fit a linear regression to this small sample by hand. Recall that we can use χ^2 to measure how good our fit is, defined by

$$\chi^2 = \sum (y_i - mx_i - c)^2,$$

and that we can find the minimum of χ^2 by differentiating it and setting it to zero. This leads us to the equations for m and c ,

$$m = \frac{\sum (x_i - \bar{x})y_i}{\sum (x_i - \bar{x})^2}, \quad c = \bar{y} - m\bar{x},$$

which minimise χ^2 .

Use these equations to calculate the m and c which minimise χ^2 for the 5 data points given above and select the correct values below:

- ☒ $m = 2, c = -0.7$
- ☐ $m = -1.8, c = 0.7$
- ☐ $m = 1.6, c = -0.5$
- ☐ $m = 1.8, c = -0.6$

5. As you have seen it can be quite a lot of effort to fit even 5 data points when doing the maths by hand. Often it's necessary to work with much larger data sets, so let's consider a new example with 50 data points. Instead of doing it by hand we'll implement a function to do the maths for us.

1 point

Run the following code block first to see the data without any kind of linear fit. The function *linfit* is being defined inside the code block. Your task is to edit the definition so that *linfit* takes the array of x data, *xdat*, and the array of y data, *ydat*, and returns the correct m and c to create a linear fit which minimises χ^2 .

The calculation for $\bar{x}$, $\bar{y}$, $\bar{y}$, $\bar{y}$, is already given. As you can see *numpy* has been imported as *np*.

```
1 # Here the function is defined
2 def linfit(xdat,ydat):
3     # Here xbar and ybar are calculated
4     xbar = np.sum(xdat)/len(xdat)
5     ybar = np.sum(ydat)/len(ydat)
6
7     # Insert calculation of m and c here. If nothing is here the data will be plotted
8     m = np.sum((xdat - xbar)*ydat)/np.sum((xdat - xbar)**2)
9     c = ybar - m*xbar
10    # Return your values as [m, c]
11    return [m, c]
12
13 # Produce the plot - don't put this in the next code block
14 line()
```

Run

Reset

Use the above code block to test your code. When you are confident that you have correctly defined the function, put it into the next codeblock and run it, being careful not to include *line()* in your answer.

```
1 # Here the function is defined
2 def linfit(xdat,ydat):
3     # Here xbar and ybar are calculated
4     xbar = np.sum(xdat)/len(xdat)
5     ybar = np.sum(ydat)/len(ydat)
6
7     # Insert calculation of m and c below
8     m = np.sum((xdat - xbar)*ydat)/np.sum((xdat - xbar)**2)
9     c = ybar - m*xbar
10    # Return your values as [m, c]
11    return [m, c]
12
13 # Don't include line() in this answer box
```

6. While it is informative to write the code ourselves, as in the previous question, in practice functions which do various types of regression are implemented in lots of programming languages. There are several of these in python.

1 point

One such example is the *scipy.stats.linregress()* method, which takes arrays of x data and y data in exactly the same way as the *linfit()* function you defined in the previous question. As an output it gives the slope m and intercept c as well as a few useful statistical measures like the standard error.

In the following code block, the x data is again stored in the *xdat* array, and the y data in the *ydat* array. Call the method *stats.linregress()* with the data arguments, and then pass the output to *line()* to plot the regression.

```
1 from scipy import stats
2
3 # Use the stats.linregress() method to evaluate regression
4 regression = stats.linregress(xdat, ydat)
5
6 line(regression)
```

Run

Reset

Hopefully it is clear that *linregress()* does everything *linfit()* did and more, without having to write it yourself!

Once you're happy that you've implemented things correctly in the above code block, repeat the same in the following code block without the last line to complete the question.

```
1 from scipy import stats
2
3 # Use the stats.linregress() method to evaluate regression
4 regression = stats.linregress(xdat, ydat)
5
6 # Don't use line(regression) in this code box
```